

WEB APPLICATION DEVELOPMENT

LAB 5 REPORT – HOMEWORK

EXERCISE 5: SEARCH FUNCTIONALITY

5.1. Update StudentDAO

Code:

```
public List<Student> searchStudents(String keyword) {
    List<Student> students = new ArrayList<>();
    String sql = "SELECT * FROM students WHERE full_name LIKE ? OR student_code LIKE ?";

    try (Connection conn = getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        String searchPattern = "%" + keyword + "%";
        pstmt.setString(1, searchPattern);
        pstmt.setString(2, searchPattern);

        ResultSet rs = pstmt.executeQuery();

        while (rs.next()) {
            Student student = new Student();
            student.setId(rs.getInt("id"));
            student.setStudentCode(rs.getString("student_code"));
            student.setFullName(rs.getString("full_name"));
            student.setEmail(rs.getString("email"));
            student.setMajor(rs.getString("major"));
            students.add(student);
        }

    } catch (SQLException e) {
        e.printStackTrace();
    }

    return students;
}
```

5.2. Add search controller method in StudentController.java

Code:

```
private void searchStudents(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    // 1. Get keyword parameter
    String keyword = request.getParameter("keyword");
    List<Student> students;

    // 2. Decide which DAO method to call
    try {
        if (keyword == null || keyword.trim().isEmpty()) {
            // 3. Get the student list
            students = studentDAO.getAllStudents();
        } else {
            // Otherwise, perform search
            students = studentDAO.searchStudents(keyword);
        }

        // 4. Set request attributes
        request.setAttribute("students", students);
        request.setAttribute("keyword", keyword);

        // 5. Forward to view
        RequestDispatcher dispatcher = request.getRequestDispatcher("/student-list.jsp");
        dispatcher.forward(request, response);

    } catch (Exception e) {
        e.printStackTrace();
        request.setAttribute("error", "Error searching students: " + e.getMessage());
        RequestDispatcher dispatcher = request.getRequestDispatcher("/student-list.jsp");
        dispatcher.forward(request, response);
    }
}
```

5.3. Update student list view in student-list.jsp

Code:

```
<!-- Search Box -->
<div class="search-box">
    <form action="student" method="get" class="search-form">

        <!-- Required hidden field -->
        <input type="hidden" name="action" value="search">

        <!-- User types search here -->
        <input
            type="text"
            name="keyword"
            placeholder="Search by name or student code..."
            value="${keyword}"
        >

        <!-- Search button -->
        <button type="submit">🔍 Search</button>

        <!-- Show Clear button only if the keyword exists -->
        <c:if test="${not empty keyword}">
            <a href="student?action=list" class="clear-btn">Clear</a>
        </c:if>
    </form>

    <!-- Optional message -->
    <c:if test="${not empty keyword}">
        <p class="search-info">Search results for: <strong>${keyword}</strong></p>
    </c:if>
</div>

@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

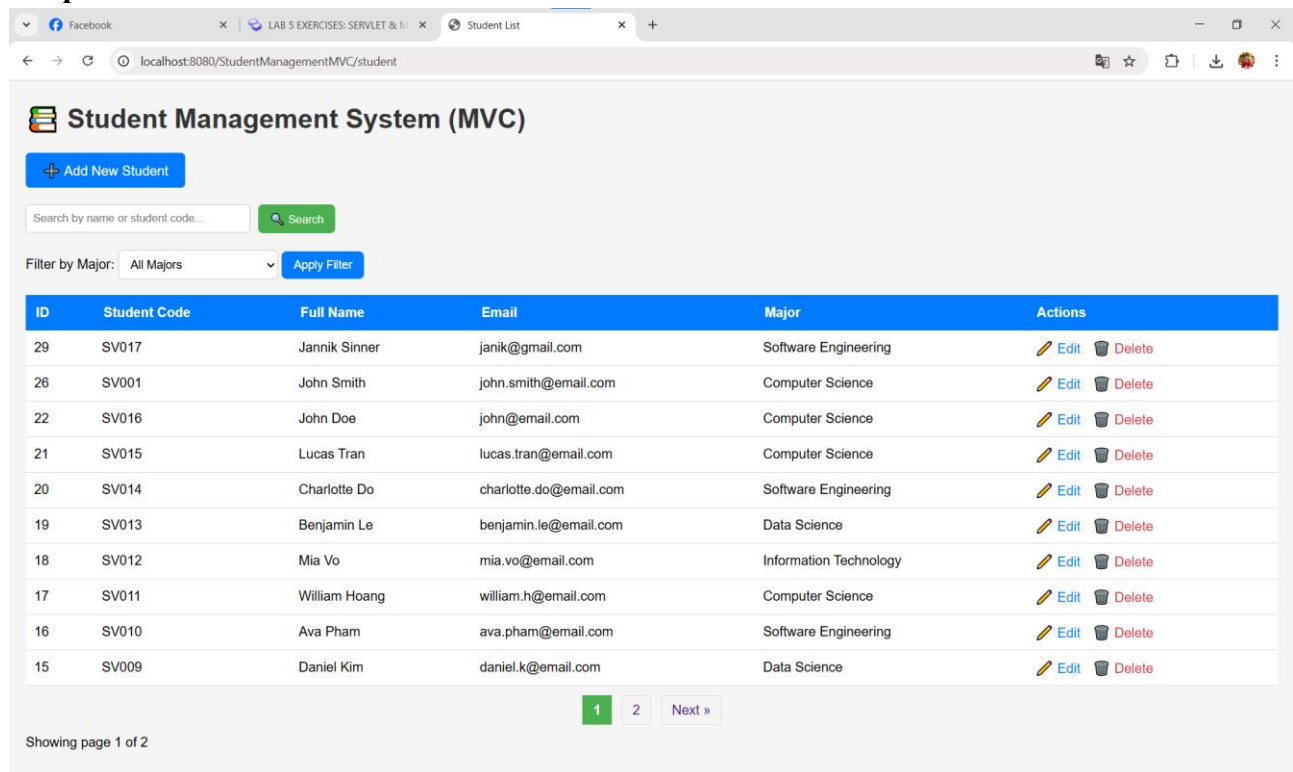
    String action = request.getParameter("action");

    if (action == null) {
        action = "list";
    }

    switch (action) {
        case "list":
            listStudents(request, response);
            break;
        case "new":
            showNewForm(request, response);
            break;
        case "edit":
            showEditForm(request, response);
            break;
        case "delete":
            deleteStudent(request, response);
            break;
        case "search":
            searchStudents(request, response);
            break;
        case "filterSort":
            filterAndSort(request, response);
            break;
        default:
            listStudents(request, response);
            break;
    }
}
```

doGet method in StudentController.java

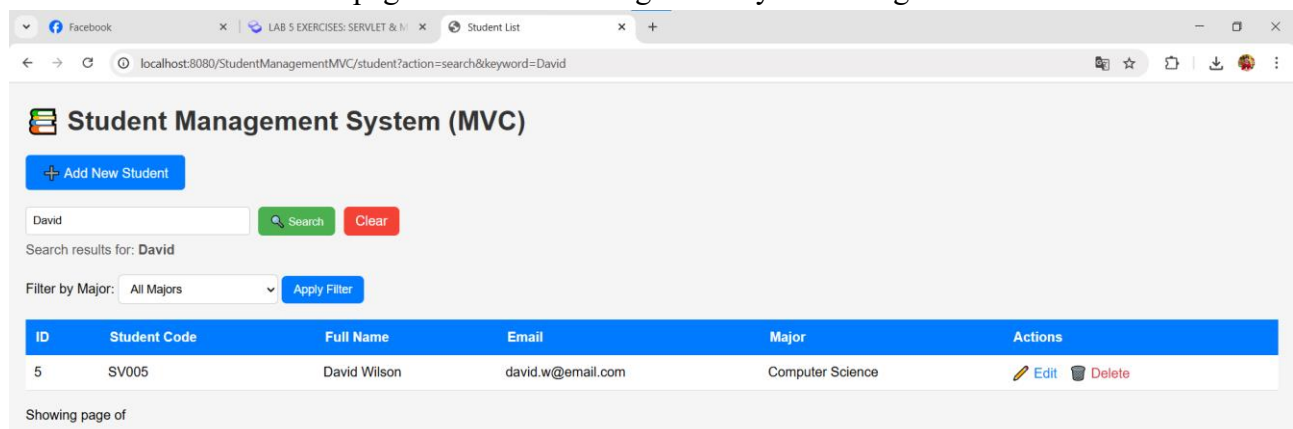
Output



The screenshot shows the homepage of the Student Management System (MVC). The interface includes a header with the system name, a button to "Add New Student", a search bar, and a filter dropdown. Below these is a table listing students with columns for ID, Student Code, Full Name, Email, Major, and Actions (Edit/Delete). The table contains 10 rows of student data. At the bottom, there is a pagination control showing "Showing page 1 of 2".

ID	Student Code	Full Name	Email	Major	Actions
29	SV017	Jannik Sinner	janik@gmail.com	Software Engineering	Edit Delete
26	SV001	John Smith	john.smith@email.com	Computer Science	Edit Delete
22	SV016	John Doe	john@email.com	Computer Science	Edit Delete
21	SV015	Lucas Tran	lucas.tran@email.com	Computer Science	Edit Delete
20	SV014	Charlotte Do	charlotte.do@email.com	Software Engineering	Edit Delete
19	SV013	Benjamin Le	benjamin.le@email.com	Data Science	Edit Delete
18	SV012	Mia Vo	mia.vo@email.com	Information Technology	Edit Delete
17	SV011	William Hoang	william.h@email.com	Computer Science	Edit Delete
16	SV010	Ava Pham	ava.pham@email.com	Software Engineering	Edit Delete
15	SV009	Daniel Kim	daniel.k@email.com	Data Science	Edit Delete

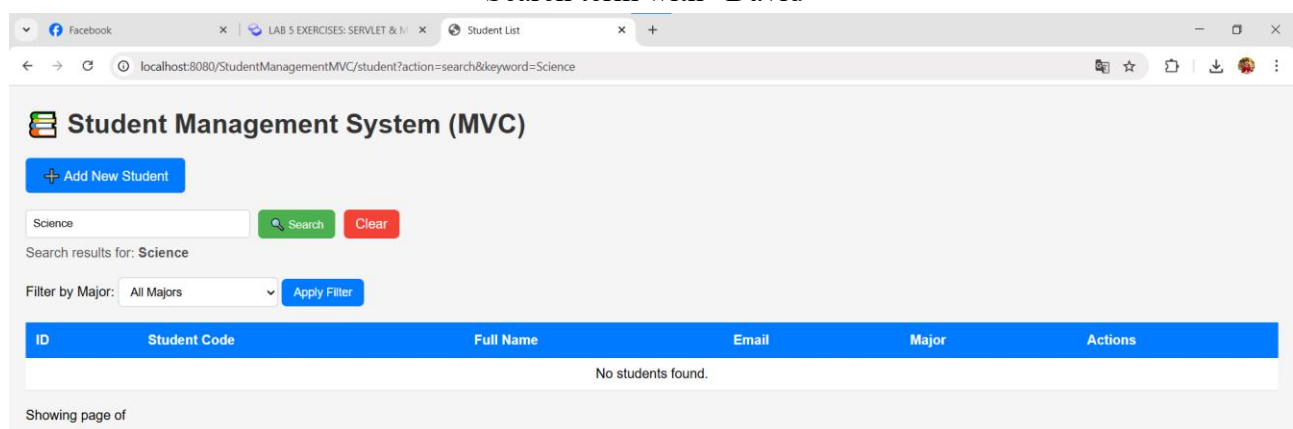
The homepage of Student Management System using MVC model



The screenshot shows the search results for the keyword "David". The search bar contains "David", and the results dropdown shows "Search results for: David". The table below displays one result for David Wilson, a student in the Computer Science major. The pagination control shows "Showing page of".

ID	Student Code	Full Name	Email	Major	Actions
5	SV005	David Wilson	david.w@email.com	Computer Science	Edit Delete

Search term with "David"



The screenshot shows the search results for the keyword "Science". The search bar contains "Science", and the results dropdown shows "Search results for: Science". The table below displays no results, with the message "No students found." shown. The pagination control shows "Showing page of".

ID	Student Code	Full Name	Email	Major	Actions
No students found.					

Search term with "Science"

Explanation about the code flow for the search functionality:

1. Browser → GET /student?action=search&keyword=abc

- The user types a keyword in the search box (e.g., “abc”) and submits the form by clicking search button.
- The .jsp file sends a GET request such as: /student?action=search&keyword=abc and send it to the server (StudentController.java file).
- 2. Server (StudentController.java file) receives the request**
 - The servlet file StudentController.java receives the request and check the “action” parameter in doGet() method by switch case. If action = search, which means that the controller decides that this request requires a search operation and goes to searchStudents() method.
 - In searchStudents(), it receives the keyword, which is typed by the user.
- 3. Controller calls StudentDAO.searchStudents(keyword)**
 - The controller does NOT handle SQL directly. Instead, it calls the method searchStudents() in StudentDAO.java file. If the keyword is empty, the controller calls DAO to display all students in the list. Otherwise, the controller calls DAO to execute the searchStudents(keyword) method in StudentDAO.java file.
 - The DAO is responsible for validating the keyword, building SQL, executing the query, and constructing Student objects.
- 4. DAO (StudentDAO.java file) validates keyword and builds SQL query. Inside searchStudents(keyword):**
 - It builds dynamic SQL: SELECT * FROM students WHERE full_name LIKE ? OR student_code LIKE ?
 - DAO creates PreparedStatement to prevent SQL injection and binds cleaned keyword such as %keyword%.
- 5. DAO executes SQL query & builds List<Student>**
 - DAO runs the query on the database (student_management).
 - For each row returned, DAO constructs a new Student object, fill attributes (id, name, email, major, etc).
- 6. DAO returns List<Student> to the controller**
 - If the keyword matches any records → list contains filtered results. Otherwise, the resulted list is empty.
 - Controller does not care. It simply gets a list.
- 7. Controller sets request attribute**

```
// 4. Set request attributes
request.setAttribute("students", students);
request.setAttribute("keyword", keyword);
```

 - Controller attaches the student list to the request as “students”.
 - It also reattaches “keyword” so the student-list.jsp file can show what was searched
- 8. Controller forwards to JSP**

```
// 5. Forward to view
RequestDispatcher dispatcher = request.getRequestDispatcher("/student-list.jsp");
dispatcher.forward(request, response);
```

 - No redirect is used because we want to preserve attributes.
- 9. JSP displays the filtered list**
 - The student-list.jsp file receives the list from the controller (server).
 - It loops through the list and prints each row on the table.
 - The table now shows only the students whose name/email/code/major matched the keyword.

EXERCISE 6: SERVER-SIDE VALIDATION

6.1. Create Validation method in StudentController.java

Code:

```
private boolean validateStudent(Student student, HttpServletRequest request) {
    boolean isValid = true;
    // Validate student code
    String code = student.getStudentCode();
    String codePattern = "[A-Z]{2}[0-9]{3}"; // 2 letters + 3+ digits

    if (code == null || code.trim().isEmpty()) {
        request.setAttribute("errorCode", "Student code is required.");
        isValid = false;
    } else if (!code.matches(codePattern)) {
        request.setAttribute("errorCode", "Invalid format. Use 2 uppercase letters followed by at least 3 digits. Example: SV001");
        isValid = false;
    }
    // TODO: Validate full name
    String name = student.getFullName();
    if (name == null || name.trim().isEmpty()) {
        request.setAttribute("errorMessage", "Full name is required.");
        isValid = false;
    } else if (name.trim().length() < 2) {
        request.setAttribute("errorMessage", "Full name must be at least 2 characters.");
        isValid = false;
    }
    // TODO: Validate email (only if provided)
    String email = student.getEmail();
    String emailPattern = "[A-Za-z0-9+_.-]+@(.+)$";

    if (email != null && !email.trim().isEmpty()) {
        if (!email.matches(emailPattern)) {
            request.setAttribute("errorMessage", "Invalid email format.");
            isValid = false;
        }
    }
    // TODO: Validate major
    String major = student.getMajor();

    if (major == null || major.trim().isEmpty()) {
        request.setAttribute("errorMajor", "Major is required.");
        isValid = false;
    }
    return isValid;
}
```

6.2. Integrate validation into insert/update in StudentController.java

Code:

```
private void insertStudent(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    // 1. Get parameters and create Student object
    String studentCode = request.getParameter("studentCode");
    String fullName = request.getParameter("fullName");
    String email = request.getParameter("email");
    String major = request.getParameter("major");

    Student student = new Student(studentCode, fullName, email, major);

    // 2. Validate
    if (!validateStudent(student, request)) {
        request.setAttribute("student", student);
        RequestDispatcher dispatcher = request.getRequestDispatcher("/student-form.jsp");
        dispatcher.forward(request, response);
        return;
    }
    // 3. If valid, proceed with insert
    if (studentDAO.addStudent(student)) {
        response.sendRedirect("student?action=list&message=Student added successfully");
    } else {
        response.sendRedirect("student?action=new&error=Failed to add student");
    }
}
```

insertStudent method in StudentController.java

```

private void updateStudent(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    // 1. Get parameters and create Student object
    int id = Integer.parseInt(request.getParameter("id"));
    String fullName = request.getParameter("fullName");
    String email = request.getParameter("email");
    String major = request.getParameter("major");
    String studentCode = request.getParameter("studentCode");

    Student student = new Student();
    student.setId(id);
    student.setFullName(fullName);
    student.setEmail(email);
    student.setMajor(major);
    student.setStudentCode(studentCode);
    // 2. Validate
    if (!validateStudent(student, request)) {
        request.setAttribute("student", student);
        RequestDispatcher dispatcher = request.getRequestDispatcher("/student-form.jsp");
        dispatcher.forward(request, response);
        return;
    }
    // 3. If valid, proceed with update
    if (studentDAO.updateStudent(student)) {
        response.sendRedirect("student?action=list&message=Student updated successfully");
    } else {
        response.sendRedirect("student?action=edit&id=" + id + "&error=Failed to update");
    }
}

```

updateStudent method in StudentController.java

6.3. Display validation errors in form (student-form.jsp)

Code:

```

<div class="form-group">
    <label for="studentCode">Student Code:</label>
    <input type="text" id="studentCode" name="studentCode"
        value="${student.studentCode}">

    <c:if test="${not empty errorCode}">
        <span class="error">${errorCode}</span>
    </c:if>
</div>

<div class="form-group">
    <label for="fullName">Full Name:</label>
    <input type="text" id="fullName" name="fullName"
        value="${student.fullName}">

    <c:if test="${not empty errorName}">
        <span class="error">${errorName}</span>
    </c:if>
</div>

<div class="form-group">
    <label for="email">Email:</label>
    <input type="text" id="email" name="email"
        value="${student.email}">

    <c:if test="${not empty errorEmail}">
        <span class="error">${errorEmail}</span>
    </c:if>
</div>

<div class="form-group">
    <label for="major">Major:</label>
    <input type="text" id="major" name="major"
        value="${student.major}">

    <c:if test="${not empty errorMajor}">
        <span class="error">${errorMajor}</span>
    </c:if>
</div>

```

Output:

+ Add New Student

Student Code:

SV001

Full Name:

Jannik Sinner

Email:

janik@email.com

Major:

Information Technology



Save

Cancel

+ Add New Student

Failed to add student

Student Code:

Full Name:

Email:

Major:



Save

Cancel

Add new student with an existing code (SV001) and the returned error message

Edit Student

Student Code:

SV017

Full Name:

Jannik Sinner

Email:

janik@

Major:

Software Engineering



Update

Cancel

Edit Student

Student Code:

SV017

Full Name:

Jannik Sinner

Email:

janik@

Invalid email format.

Major:

Software Engineering



Update

Cancel

Edit student with invalid email format and the returned error message

Explanation about the code flow for the validation:

1. Browser → User submits the form
 - The user fills in all information (Student code, full name, email, major) and then clicks save or update. When the user clicked, a POST request is sent to controller (StudentController.java file): /student?action=insert or /student?action=update.
2. StudentController receives the request:
 - Depending on action, the controller calls insertStudent() or updateStudent() method
3. Controller extracts form data
 - A Student object is created with all information typed by user.
4. Controller runs validation BEFORE doing anything else
 - It will validate all information that user typed. If validation fails,
 - + It stores entered data → `${student}` in JSP.
 - + It stores error messages → `${errorCode}`, `${errorName}`, `${errorEmail}`, `${errorMajor}`.
 - + It forwards back to the form without touching the database and stops the method, which prevents invalid data from ever reaching the DAO.
5. The validateStudent() method checks each field.
 - It creates a flag called isValid and assigned it to the truth.
 - This flag will be false if a validation of all fields fails. Inside this method, it validates the

- + Student code, which cannot miss and must be followed the format that 2 uppercase letter and 3 or more digits by using the regular expression (regex).
 - + Email, which must be followed the email format by using the regex. If the input is empty, it is accepted since the email is optional and skip the validation.
 - + Full name and major cannot be missing. If missing, the error message will be printed to announce the user to input that field.
 - Finally, the method returns the value of the flag “isValid”.
6. Controller receives validation result
- If the value of the flag is false, the controller back to the form (student-form.jsp file), input stays in the form, and errors appear next to each field, which is only visible when validation fails. It also prevents inserting data into the database and redirecting.
 - If the value of the flag is true, the controller calls DAO for inserting via addStudent() method or updating via updateStudent() method. These operations are now safe because the data is already validated.
7. Controller redirects with success message:
- ```
response.sendRedirect("student?action=list&message=Added successfully");
```

## EXERCISE 7: SORTING & FILTERING

### 7.1. Add Sort & Filter Methods to DAO

Code:

```
public List<Student> getStudentsFiltered(String major, String sortBy, String order) {
 List<Student> students = new ArrayList<>();
 // Validate sorting inputs
 String validSortBy = validateSortBy(sortBy);
 String validOrder = validateOrder(order);
 // Base query
 String sql = "SELECT * FROM students WHERE 1=1";
 // If major filter is active
 if (major != null && !major.trim().isEmpty()) {
 sql += " AND major = ?";
 }
 // Add sorting
 sql += " ORDER BY " + validSortBy + " " + validOrder;
 try (Connection conn = getConnection();
 PreparedStatement pstmt = conn.prepareStatement(sql)) {
 int index = 1;
 // Bind major parameter only if provided
 if (major != null && !major.trim().isEmpty()) {
 pstmt.setString(index++, major);
 }
 ResultSet rs = pstmt.executeQuery();
 while (rs.next()) {
 Student s = new Student();
 s.setId(rs.getInt("id"));
 s.setStudentCode(rs.getString("student_code"));
 s.setFullName(rs.getString("full_name"));
 s.setEmail(rs.getString("email"));
 s.setMajor(rs.getString("major"));
 students.add(s);
 }
 } catch (SQLException e) {
 e.printStackTrace();
 }
 return students;
}
```



```

// Validate sortBy parameter
private String validateSortBy(String sortBy) {

 String[] validColumns = {
 "id", "student_code", "full_name", "email", "major"
 };

 if (sortBy == null) return "id";

 for (String col : validColumns) {
 if (col.equalsIgnoreCase(sortBy)) {
 return col;
 }
 }

 return "id"; // fallback
}

// Validate order parameter
private String validateOrder(String order) {
 if (order == null) {
 return "ASC";
 }
 if ("desc".equalsIgnoreCase(order)) {
 return "DESC";
 }
 return "ASC"; // default
}

```

## 7.2. Add controller methods

Code:

```

private void filterAndSort(HttpServletRequest request, HttpServletResponse response)
 throws ServletException, IOException {
 String major = request.getParameter("major");
 String sortBy = request.getParameter("sortBy");
 String order = request.getParameter("order");

 List<Student> students = studentDAO.getStudentsFiltered(major, sortBy, order);

 request.setAttribute("students", students);
 request.setAttribute("selectedMajor", major);
 request.setAttribute("sortBy", sortBy);
 request.setAttribute("order", order);

 RequestDispatcher dispatcher = request.getRequestDispatcher("/student-list.jsp");
 dispatcher.forward(request, response);
}

```

## 7.3. Update view sort and filter UI

Code:

```

<div class="filter-box">
 <form action="student" method="get">
 <input type="hidden" name="action" value="filterSort">
 <label>Filter by Major:</label>
 <select name="major">
 <option value="">All Majors</option>
 <option value="Computer Science">Computer Science</option>
 <option value="Information Technology">Information Technology</option>
 <option value="Software Engineering">Software Engineering</option>
 <option value="Business Administration">Business Administration</option>
 </select>
 <button type="submit">Apply Filter</button>
 <c:if test="${not empty selectedMajor}">
 Clear Filter
 </c:if>
 </form>
</div>

```

```

<thead>
 <tr>
 <th>

 ID

 <c:if test="${sortBy == 'id'}">
 ${order == 'asc' ? '▲' : '▼'}
 </c:if>
 </th>

 <th>

 Student Code

 <c:if test="${sortBy == 'student_code'}">
 ${order == 'asc' ? '▲' : '▼'}
 </c:if>
 </th>

 <th>

 Full Name

 <c:if test="${sortBy == 'full_name'}">
 ${order == 'asc' ? '▲' : '▼'}
 </c:if>
 </th>

 <th>

 Email

 <c:if test="${sortBy == 'email'}">
 ${order == 'asc' ? '▲' : '▼'}
 </c:if>
 </th>

 <th>

 Major

 <c:if test="${sortBy == 'major'}">
 ${order == 'asc' ? '▲' : '▼'}
 </c:if>
 </th>

```

## Output:

Student Management System (MVC)

[Add New Student](#)

Search by name or student code... [Search](#)

Filter by Major: All Majors [Apply Filter](#) [Clear Filter](#)

| ID | Student Code | Full Name     | Email                 | Major            | Actions                                     |
|----|--------------|---------------|-----------------------|------------------|---------------------------------------------|
| 5  | SV005        | David Wilson  | david.w@email.com     | Computer Science | <a href="#">Edit</a> <a href="#">Delete</a> |
| 14 | SV008        | Sophia Tran   | sophia.tran@email.com | Computer Science | <a href="#">Edit</a> <a href="#">Delete</a> |
| 17 | SV011        | William Hoang | william.h@email.com   | Computer Science | <a href="#">Edit</a> <a href="#">Delete</a> |
| 21 | SV015        | Lucas Tran    | lucas.tran@email.com  | Computer Science | <a href="#">Edit</a> <a href="#">Delete</a> |
| 22 | SV016        | John Doe      | john@email.com        | Computer Science | <a href="#">Edit</a> <a href="#">Delete</a> |
| 26 | SV001        | John Smith    | john.smith@email.com  | Computer Science | <a href="#">Edit</a> <a href="#">Delete</a> |

Filtering by major = "Computer Science"

Facebook LAB 5 EXERCISES: SERVLET & JSP Student List

localhost:8080/StudentManagementMVC/student?action=filterSort&major=Information+Technology

## Student Management System (MVC)

[Add New Student](#)

Search by name or student code... [Search](#)

Filter by Major: All Majors [Apply Filter](#) [Clear Filter](#)

| ID | Student Code | Full Name     | Email              | Major                  | Actions                                     |
|----|--------------|---------------|--------------------|------------------------|---------------------------------------------|
| 2  | SV002        | Emily Johnson | emily.j@email.com  | Information Technology | <a href="#">Edit</a> <a href="#">Delete</a> |
| 12 | SV006        | Olivia Nguyen | olivia.n@email.com | Information Technology | <a href="#">Edit</a> <a href="#">Delete</a> |
| 18 | SV012        | Mia Vo        | mia.vo@email.com   | Information Technology | <a href="#">Edit</a> <a href="#">Delete</a> |

Filtering by major = "Information Technology"

Facebook LAB 5 EXERCISES: SERVLET & JSP Student List

localhost:8080/StudentManagementMVC/student?action=filterSort&sortBy=student\_code&order=asc

## Student Management System (MVC)

[Add New Student](#)

Search by name or student code... [Search](#)

Filter by Major: All Majors [Apply Filter](#)

| ID | Student Code | Full Name     | Email                 | Major                  | Actions                                     |
|----|--------------|---------------|-----------------------|------------------------|---------------------------------------------|
| 26 | SV001        | John Smith    | john.smith@email.com  | Computer Science       | <a href="#">Edit</a> <a href="#">Delete</a> |
| 2  | SV002        | Emily Johnson | emily.j@email.com     | Information Technology | <a href="#">Edit</a> <a href="#">Delete</a> |
| 3  | SV003        | Michael Brown | michael.b@email.com   | Software Engineering   | <a href="#">Edit</a> <a href="#">Delete</a> |
| 4  | SV004        | Sarah Davis   | sarah.d@email.com     | Data Science           | <a href="#">Edit</a> <a href="#">Delete</a> |
| 5  | SV005        | David Wilson  | david.w@email.com     | Computer Science       | <a href="#">Edit</a> <a href="#">Delete</a> |
| 12 | SV006        | Olivia Nguyen | olivia.n@email.com    | Information Technology | <a href="#">Edit</a> <a href="#">Delete</a> |
| 13 | SV007        | James Lee     | james.lee@email.com   | Software Engineering   | <a href="#">Edit</a> <a href="#">Delete</a> |
| 14 | SV008        | Sophia Tran   | sophia.tran@email.com | Computer Science       | <a href="#">Edit</a> <a href="#">Delete</a> |
| 15 | SV009        | Daniel Kim    | daniel.k@email.com    | Data Science           | <a href="#">Edit</a> <a href="#">Delete</a> |
| 16 | SV010        | Ava Pham      | ava.pham@email.com    | Software Engineering   | <a href="#">Edit</a> <a href="#">Delete</a> |
| 17 | SV011        | William Hoang | william.h@email.com   | Computer Science       | <a href="#">Edit</a> <a href="#">Delete</a> |
| 18 | SV012        | Mia Vo        | mia.vo@email.com      | Information Technology | <a href="#">Edit</a> <a href="#">Delete</a> |

Sorted via column "student code" in ascending order

Facebook LAB 5 EXERCISES: SERVLET & JSP Student List

localhost:8080/StudentManagementMVC/student?action=filterSort&sortBy=major&order=asc

## Student Management System (MVC)

[Add New Student](#)

Search by name or student code... [Search](#)

Filter by Major: All Majors [Apply Filter](#)

| ID | Student Code | Full Name     | Email                 | Major                  | Actions                                     |
|----|--------------|---------------|-----------------------|------------------------|---------------------------------------------|
| 5  | SV005        | David Wilson  | david.w@email.com     | Computer Science       | <a href="#">Edit</a> <a href="#">Delete</a> |
| 14 | SV008        | Sophia Tran   | sophia.tran@email.com | Computer Science       | <a href="#">Edit</a> <a href="#">Delete</a> |
| 17 | SV011        | William Hoang | william.h@email.com   | Computer Science       | <a href="#">Edit</a> <a href="#">Delete</a> |
| 21 | SV015        | Lucas Tran    | lucas.tran@email.com  | Computer Science       | <a href="#">Edit</a> <a href="#">Delete</a> |
| 22 | SV016        | John Doe      | john@email.com        | Computer Science       | <a href="#">Edit</a> <a href="#">Delete</a> |
| 26 | SV001        | John Smith    | john.smith@email.com  | Computer Science       | <a href="#">Edit</a> <a href="#">Delete</a> |
| 4  | SV004        | Sarah Davis   | sarah.d@email.com     | Data Science           | <a href="#">Edit</a> <a href="#">Delete</a> |
| 15 | SV009        | Daniel Kim    | daniel.k@email.com    | Data Science           | <a href="#">Edit</a> <a href="#">Delete</a> |
| 19 | SV013        | Benjamin Le   | benjamin.le@email.com | Data Science           | <a href="#">Edit</a> <a href="#">Delete</a> |
| 2  | SV002        | Emily Johnson | emily.j@email.com     | Information Technology | <a href="#">Edit</a> <a href="#">Delete</a> |
| 12 | SV006        | Olivia Nguyen | olivia.n@email.com    | Information Technology | <a href="#">Edit</a> <a href="#">Delete</a> |
| 18 | SV012        | Mia Vo        | mia.vo@email.com      | Information Technology | <a href="#">Edit</a> <a href="#">Delete</a> |
| 3  | SV003        | Michael Brown | michael.b@email.com   | Software Engineering   | <a href="#">Edit</a> <a href="#">Delete</a> |

Sorted via column "major" in ascending order

### Explanation about the code flow for the sort and filter methods:

1. Browser sends request with search or sorting parameters
  - User selects major from dropdown menu or clicks column headers such as Student Code or Full Name.
  - The .jsp file create URL link such as  
student?action=filterSort&major=Computer+Science  
student?action=filterSort&sortBy=major&order=asc  
Then, it sends GET request with filter or sort parameters
2. Controller receives the request
  - The servlet file StudentController.java receives the request and check the “action” parameter in doGet() method by switch case. If action = filterSort, which means that the controller decides that this request requires to go to filterAndSort()method.
  - In filterAndSort(), it receives keyword, sortBy and order, which is input by the user. After that, it creates Student object and calls DAO to execute the filter.  
List<Student> students = studentDAO.getStudentsFiltered(major, sortBy, order);
3. DAO receives parameters and begins validation
  - It validates the inputs to make sure that all inputs are not empty to prevents null value, and SQL injection.
4. DAO builds safe, validated SQL query
  - After validation, SQL is built using only trusted values: SELECT \* FROM students WHERE 1=1 AND major = ? ORDER BY ASC
  - Using PreparedStatement parameters for a safe from injection.
  - sortBy/order are validated → impossible for injection.
5. DAO binds parameters and executes query
  - If keyword was empty, this becomes “%%” and matches all rows. Then, it return full list of students. Otherwise, it becomes “%keyword%”.
6. Controller sets request attribute

```
request.setAttribute("students", students);
request.setAttribute("selectedMajor", major);
request.setAttribute("sortBy", sortBy);
request.setAttribute("order", order);
```

  - Controller attaches the student list to request as “students”.
  - Preserves UI state: selectedMajor, sortBy, order
  - Forwards to the student-list.jsp file to display the result
7. Controller forwards to JSP

```
RequestDispatcher dispatcher = request.getRequestDispatcher("/student-list.jsp");
dispatcher.forward(request, response);
```
8. JSP displays the result list
  - Shows students in table with current sort indicators (▲ or ▼).
  - Preserves selected major in dropdown
  - Maintains sort state in column headers
  - Displays filtered, sorted data to user

### EXERCISE 8: PAGINATION

#### 8.1. Add pagination methods to DAO

Code:

```

 public int getTotalStudents() {
 String sql = "SELECT COUNT(*) FROM students";

 try (Connection conn = getConnection();
 Statement stmt = conn.createStatement();
 ResultSet rs = stmt.executeQuery(sql)) {

 if (rs.next()) {
 return rs.getInt(1);
 }

 } catch (SQLException e) {
 e.printStackTrace();
 }

 return 0;
 }

 public List<Student> getStudentsPaginated(int offset, int recordsPerPage) {
 List<Student> students = new ArrayList<>();
 String sql = "SELECT * FROM students ORDER BY id DESC LIMIT ? OFFSET ?";

 try (Connection conn = getConnection();
 PreparedStatement pstmt = conn.prepareStatement(sql)) {

 pstmt.setInt(1, recordsPerPage);
 pstmt.setInt(2, offset);

 ResultSet rs = pstmt.executeQuery();

 while (rs.next()) {
 Student student = new Student();
 student.setId(rs.getInt("id"));
 student.setStudentCode(rs.getString("student_code"));
 student.setFullName(rs.getString("full_name"));
 student.setEmail(rs.getString("email"));
 student.setMajor(rs.getString("major"));
 student.setCreatedAt(rs.getTimestamp("created_at"));

 students.add(student);
 }

 } catch (SQLException e) {
 e.printStackTrace();
 }

 return students;
 }
 }

```

## 8.2. Add pagination controller logic

Code:

```

 private void listStudents(HttpServletRequest request, HttpServletResponse response)
 throws ServletException, IOException {
 // Get current page (default to 1)
 String pageParam = request.getParameter("page");
 int currentPage = (pageParam != null) ? Integer.parseInt(pageParam) : 1;
 if (currentPage < 1) currentPage = 1;

 // Records per page
 int recordsPerPage = 10;

 // Calculate offset
 int offset = (currentPage - 1) * recordsPerPage;

 // Get data
 List<Student> students = studentDAO.getStudentsPaginated(offset, recordsPerPage);
 int totalRecords = studentDAO.getTotalStudents();

 // Calculate total pages
 int totalPages = (int) Math.ceil((double) totalRecords / recordsPerPage);
 if (currentPage > totalPages && totalPages > 0) {
 currentPage = totalPages;
 }

 // Set attributes
 request.setAttribute("students", students);
 request.setAttribute("currentPage", currentPage);
 request.setAttribute("totalPages", totalPages);

 RequestDispatcher dispatcher = request.getRequestDispatcher("/student-list.jsp");
 dispatcher.forward(request, response);
 }

```

### 8.3. Add pagination controls to view

Code:

```
<div class="pagination">
 <!-- Previous button -->
 <c:if test="${currentPage > 1}">
 << Previous
 </c:if>

 <!-- Page numbers -->
 <c:forEach begin="1" end="${totalPages}" var="i">
 <c:choose>
 <c:when test="${i == currentPage}">
 ${i}
 </c:when>
 <c:otherwise>
 ${i}
 </c:otherwise>
 </c:choose>
 </c:forEach>

 <!-- Next button -->
 <c:if test="${currentPage < totalPages}">
 Next >>
 </c:if>
</div>
<p>Showing page ${currentPage} of ${totalPages}</p>
```

Output:

The screenshot shows a web browser window displaying the 'Student Management System (MVC)'. The page has a header with the title and a navigation bar with an 'Add New Student' button. Below the header is a search bar with the text 'Search by name or student code...' and a 'Search' button. There is also a filter section with a dropdown menu set to 'All Majors' and an 'Apply Filter' button. The main content area features a table with the following columns: ID, Student Code, Full Name, Email, Major, and Actions. The table contains 12 rows of student data. At the bottom of the table, there are pagination controls showing '1' and '2' as page numbers, and a 'Next »' button. Below the pagination controls, it says 'Showing page 1 of 2'.

ID	Student Code	Full Name	Email	Major	Actions
29	SV017	Jannik Sinner	janik@email.com	Software Engineering	<a href="#">Edit</a> <a href="#">Delete</a>
26	SV001	John Smith	john.smith@email.com	Computer Science	<a href="#">Edit</a> <a href="#">Delete</a>
22	SV016	John Doe	john@email.com	Computer Science	<a href="#">Edit</a> <a href="#">Delete</a>
21	SV015	Lucas Tran	lucas.tran@email.com	Computer Science	<a href="#">Edit</a> <a href="#">Delete</a>
20	SV014	Charlotte Do	charlotte.do@email.com	Software Engineering	<a href="#">Edit</a> <a href="#">Delete</a>
19	SV013	Benjamin Le	benjamin.le@email.com	Data Science	<a href="#">Edit</a> <a href="#">Delete</a>
18	SV012	Mia Vo	mia.vo@email.com	Information Technology	<a href="#">Edit</a> <a href="#">Delete</a>
17	SV011	William Hoang	william.h@email.com	Computer Science	<a href="#">Edit</a> <a href="#">Delete</a>
16	SV010	Ava Pham	ava.pham@email.com	Software Engineering	<a href="#">Edit</a> <a href="#">Delete</a>
15	SV009	Daniel Kim	daniel.k@email.com	Data Science	<a href="#">Edit</a> <a href="#">Delete</a>

#### Explanation about the code flow for the pagination

1. Browser → GET /student?action=list
  - The user loads the student list page or clicks: “Previous”, “Next” or a page number (1, 2, 3...), the .jsp file creates an URL such as student?action=list&page=1. If page is not included, default is page 1.
  - Then, the browser sends GET request with the page parameter.



2. StudentController receives the request
  - The servlet file StudentController.java receives the request and check the “action” parameter in doGet() method by switch case. If action = list, which means that the controller decides that this request requires to go to listStudents() method.
3. Controller reads the "page" parameter
  - The listStudents() method receivers “page” parameter, which choosing by the user, executed by String pageParam = request.getParameter("page");. If this parameter is assigned, use that value stored in that. Otherwise, assigned it to 1.
4. Controller defines LIMIT and OFFSET
  - Assigning 10 records per page, which is a limit record for each page.
  - Calculating the offset: offset = (currentPage - 1) \* recordsPerPage.
5. Controller calls the DAO
  - The controller creates a Student object and calls DAO for execution, which will get 10 students starting from offset and counting the number of students in the list via getTotalStudents() method and SQL query: SELECT COUNT(\*) FROM students.
6. DAO runs SQL with LIMIT and OFFSET
  - Data query: SELECT \* FROM students ORDER BY id DESC LIMIT ? OFFSET ? which returns only the rows needed for that page.
  - Uses PreparedStatement for safe parameter binding
7. Controller calculates the number of pages by the formula:  

$$\text{int totalPages} = (\text{int}) \text{Math.ceil}((\text{double}) \text{totalRecords} / \text{recordsPerPage});$$
8. Controller sets attributes for the JSP
 

```
//Set attributes
request.setAttribute("students", students);
request.setAttribute("currentPage", currentPage);
request.setAttribute("totalPages", totalPages);
```

  - These values will be used in the JSP to:
    - + Display 10 current students
    - + Render page number buttons
    - + Disable or enable Next/Previous
9. Controller forwards to student-list.jsp
 

```
RequestDispatcher dispatcher = request.getRequestDispatcher("/student-list.jsp");
dispatcher.forward(request, response);
```
10. JSP displays the current page of students
  - Shows only 10 students in table
  - Displays pagination controls:
    - + Next button (enabled, links to page 2)
    - + Page numbers 1-2 with page 1 highlighted
    - + Next button (enabled, links to page 2)
  - Shows "Showing page 1 of 2" message